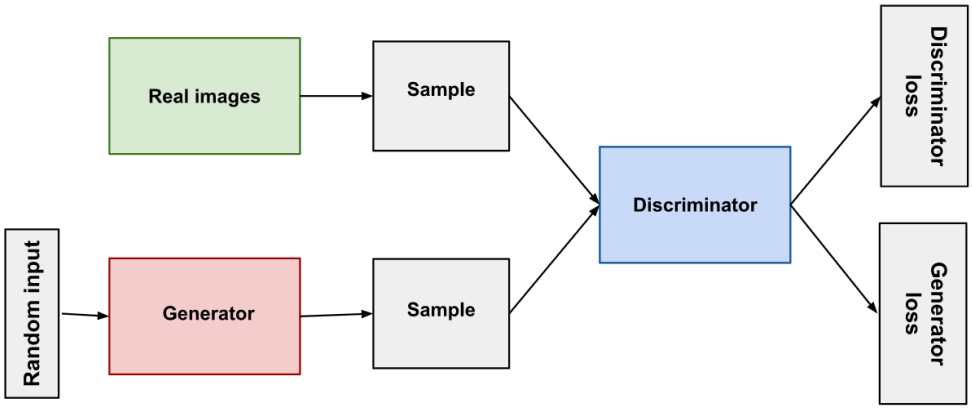
Text To Face Generation

This project aims to build a system that generates faces from text inputs. This type of system can be used by law enforcement authorities to generate the faces of criminals through textual descriptions. This project uses GANs to generate faces.

What are GANs?

Generative Adversarial Networks, or GANs, are a powerful class of neural networks mainly used for unsupervised learning. GANs as a concept was first introduced by Ian Goodfellow in 2014. GANs consist of two neural networks pitted against each other to generate new instances of data. This data could be images, voice, or video.

How do GANs work?



The generator learns to generate a fake sample from a given random input. The discriminator learns to distinguish between the samples generated by the generator and real data. Basically, the generator tries to fool the discriminator by learning to create samples as close as possible to real samples.

The discriminator loss penalises the discriminator for misclassifying real images as fake and vice versa. The discriminator updates its weights using backpropagation from discriminator loss. The generator loss is derived from the discriminator's classification of an image as real or fake. The generator loss is used to update the weights of the generator via backpropagation.

GANs Implementation

The major steps involved in implementation are:

1. **Data Loading**
   Since we have to use a custom dataset, a custom Dataset class is created. Moreover, image transformation (resizing to uniform dimensions, normalisation of images) is also used. This model considers the colour of the lips as a classifying attribute. Each unique colour in csv file is assigned a number.

2. **Discriminator and Generator Network**

The discriminator uses CNN to extract features from the image. A stride of 2 is used to reduce the size of the image. Leaky ReLU is used as it allows gradients from the discriminator to flow stronger into the generator.

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 64, 32, 32]           3,072
       BatchNorm2d-2           [-1, 64, 32, 32]             128
         LeakyReLU-3           [-1, 64, 32, 32]               0
            Conv2d-4          [-1, 128, 16, 16]         131,072
       BatchNorm2d-5          [-1, 128, 16, 16]             256
         LeakyReLU-6          [-1, 128, 16, 16]               0
            Conv2d-7            [-1, 256, 8, 8]         524,288
       BatchNorm2d-8            [-1, 256, 8, 8]             512
         LeakyReLU-9            [-1, 256, 8, 8]               0
           Conv2d-10            [-1, 512, 4, 4]       2,097,152
      BatchNorm2d-11            [-1, 512, 4, 4]           1,024
        LeakyReLU-12            [-1, 512, 4, 4]               0
           Conv2d-13              [-1, 1, 1, 1]           8,192
          Flatten-14                    [-1, 1]               0
          Sigmoid-15                    [-1, 1]               0
================================================================
Total params: 2,765,696
Trainable params: 2,765,696
Non-trainable params: 0
----------------------------------------------------------------
```

Discriminator Architecture Summary

The generator is fed a vector of random numbers as a seed for generating images.

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
   ConvTranspose2d-1            [-1, 512, 4, 4]       1,048,576
       BatchNorm2d-2            [-1, 512, 4, 4]           1,024
              ReLU-3            [-1, 512, 4, 4]               0
   ConvTranspose2d-4            [-1, 256, 8, 8]       2,097,152
       BatchNorm2d-5            [-1, 256, 8, 8]             512
              ReLU-6            [-1, 256, 8, 8]               0
   ConvTranspose2d-7          [-1, 128, 16, 16]         524,288
       BatchNorm2d-8          [-1, 128, 16, 16]             256
              ReLU-9          [-1, 128, 16, 16]               0
  ConvTranspose2d-10           [-1, 64, 32, 32]         131,072
      BatchNorm2d-11           [-1, 64, 32, 32]             128
             ReLU-12           [-1, 64, 32, 32]               0
  ConvTranspose2d-13            [-1, 3, 64, 64]           3,072
             Tanh-14            [-1, 3, 64, 64]               0
================================================================
Total params: 3,806,080
Trainable params: 3,806,080
Non-trainable params: 0
----------------------------------------------------------------
```

Generator Architecture Summary

3. **Training Generator and Discriminator**

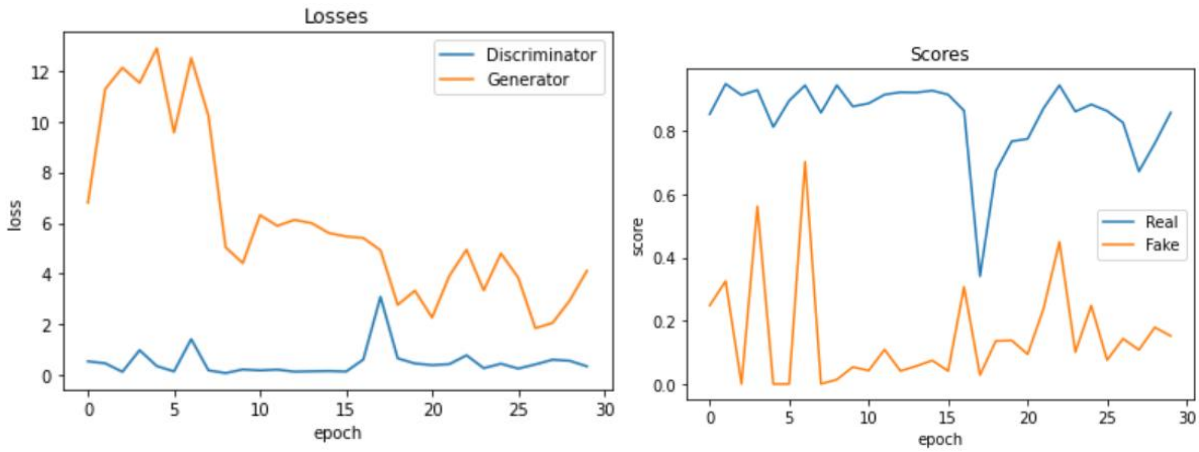
In the discriminator, binary cross-entropy loss function is used for differentiating between real and fake images. The discriminator is trained by first giving it a batch of real images (the target label is 1), and loss is calculated according to its output. Then, a batch of fake images (generated by the generator) is given (the target label is 0), and the loss is calculated. Both of these losses are added together to give the final loss, and accordingly, the weights are updated.

The generator generates a batch of images, which are passed into the discriminator (target label 1). The target label is 1, as the generator aims to fool the discriminator. The loss is calculated, and the weights are updated so that the generator can generate better images so as to fool the discriminator.

The discriminator is trained while the generator's weights remain constant. The discriminator is trained before the generator, as it will be difficult to fool a good discriminator, resulting in the generation of a good generator.

Images generated by the generator after 30 epochs

Plotting Results



We can see that with each epoch, the generator loss is decreasing, which means our generator is getting better.